Intelligenza artificiale

Giorgio Poluzzi

4 novembre 2026

1 AI-problem-solving

1.1 Agents and environments

Gli agenti sono entità che percepiscono l'ambiente. La funzione degli agenti è mappare le percezioni in azioni.

$$f : P^* \to A$$

Il programma dell'agente viene elaborato su una struttura fisica producendo f.

Se un agente possiede $|P|$ percezioni, allora il numero di entrate possibili sarà

$$\sum_{t=1}^{T} |P|^t$$

Lo scopo dell'AI è quello di progettare piccoli programmi che rappresentino funzioni complesse.

Esempio

```
function Reflex-Vacuum-Agent([locatiom.status]) returns an action
         if status = Dirty then return Suck
         else if location = A then return Right
         else if location = B then return Left
```

1.2 Rationality

Un agente razionale sceglie quale azione massimizza il valore atteso della misura della performance, data la sequenza di percezioni passate.

Le percezioni non devono essere provviste di tutte le informazioni rilevanti.

I risultati delle azioni potrebbero non essere sempre prevedibili.

La razionalizzazione dei risultati delle azioni dipende dall'esplorazione, dall'autonomia e da ciò che l'agente ha appreso.

Esercizio

- L'ambiente è composto da tre stanze (A, B, C) e due robot (r1, r2).
- Il robot r1 può pattugliare le stanze A e B, mentre r2 può pattugliare le stanze B e C.
- Alla partenza, r1 si trova nella stanza A e r2 si trova nella stanza C.

- Il tempo di viaggio necessario per spostarsi tra le stanze è pari a zero.
- La misura di prestazione da ottimizzare è la minimizzazione della somma dell'ozio medio (average idleness) di tutte le stanze.
- L'ozio medio è definito come la somma degli intervalli di tempo in cui una stanza non è visitata da alcun robot, divisa per il tempo totale.

L'esercizio chiede di identificare quale sarebbe un comportamento razionale per questo specifico ambiente.

Dal momento che un robot può pattugliare contemporaneamente solo una stanza, ci sarà sempre una stanza vuota.

Bisogna minimizzare l'ozio totale, minimizzando il numero di stanze vuote nel tempo.

Se i robot pattugliano una stanza ciascuno, la stanza B sarà vuota, ma se entrambi pattugliano B, allora ci saranno due stanze vuote.

Un comportamento razionale sarebbe quello di far pattugliare A ad r1 e C a r2.

1.3 Peas

Per progettare un agente razionale, bisogna specificare la missione dell'ambiente.

Il tipo di ambiente determina in gran parte l'architettura dell'agente.

1.4 Problem types

I tipi di problema possibili sono:

- Deterministico, totalmente osservabile → single-state problem
- Non osservabile → conformant problem
- Non deterministico, parzialmente osservabile → contingency problem
- Stato dello spazio non conosciuto → exploration problem

1.5 Problem-solving agents

I goal-based agents sono agenti che scelgono le azioni da compiere in base allo stato dell'ambiente, cercano una soluzione e eseguono la soluzione ignorando le percezioni (approccio offline).

```
function Simple-Problem-Solving-Agent(percept) returns an action
static: seq, an action sequence, initially empty
        state, some description of the current world state
        goal, a goal, initially null
        problem, a problema formulation
state <- Update-State(state, percept)
if seq is empty then
   goal <- Formulate-Goal(state)
   problem <- Formulate-Problem(state,goal)
   seq<-Search(problem)
action <- First(seq)
seq <- Rest(seq)
return action
```

Esempio

Holiday in Romania:

Formulare un goal:

- andare a Bucharest

Formulare un problema:

- states: various cities
- actions: drive between cities

Trovare una soluzione:

- trovare un percorso che metta le città in sequenza

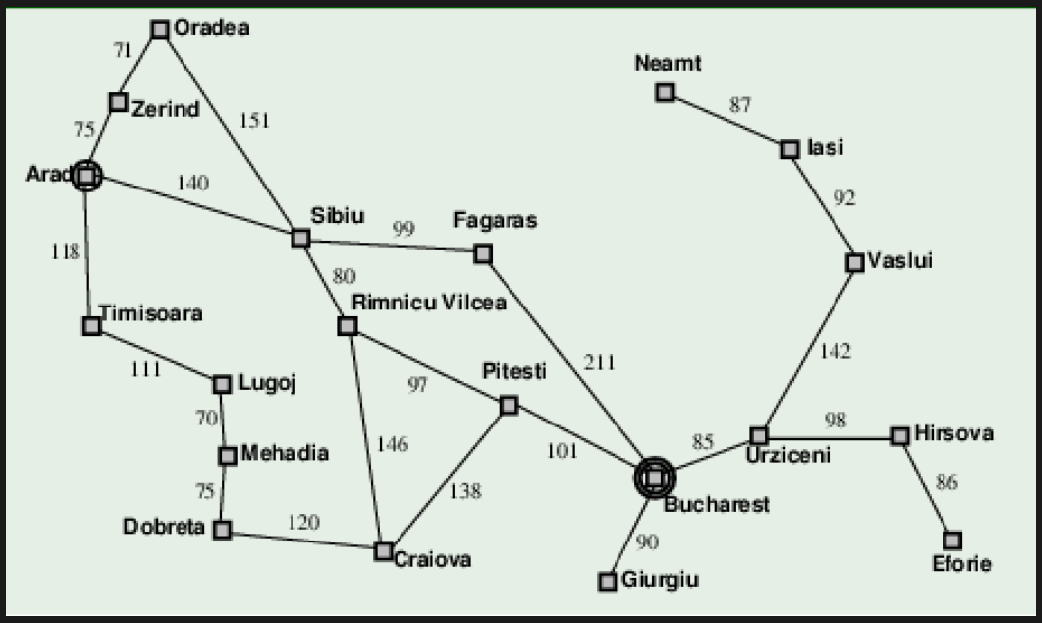


Figura 1:

```
initial state = Arad
successor function S(x) = set of action-state pairs
                          {<Arad->Zerind,Zerind>,...}
goalt test
      explicit x=Bucharest
      implicit NoDirt(x)
path cost(additive)
     sum of distances, number of actions excecuted
     c(x,a,y) is the step cost (>=0)
solution
     sequence of actions leading from initial state to goal
```

1.6 Tree search algorithm

```
function Tree-Search(problem,strategy) return solution or failure
initialize the search tree using the initiale state of problem
loop do
     if no candidates for expansion then return failure
choose a leaf node for expansion according to strategy
```

```
if node contains a goal state then return a solution
add successor nodes to the search tree (expansion)
end
```

1.7 State vs Nodes

Uno stato è una rappresentazione di una configurazione fisica dell'ambiente

Un nodo è un dato strutturale che costituisce una parte dell'alber di ricerca, e include parent, action, children, depth, path cost... .

Gli stati invece non contengono nodi figli, parenti ecc...

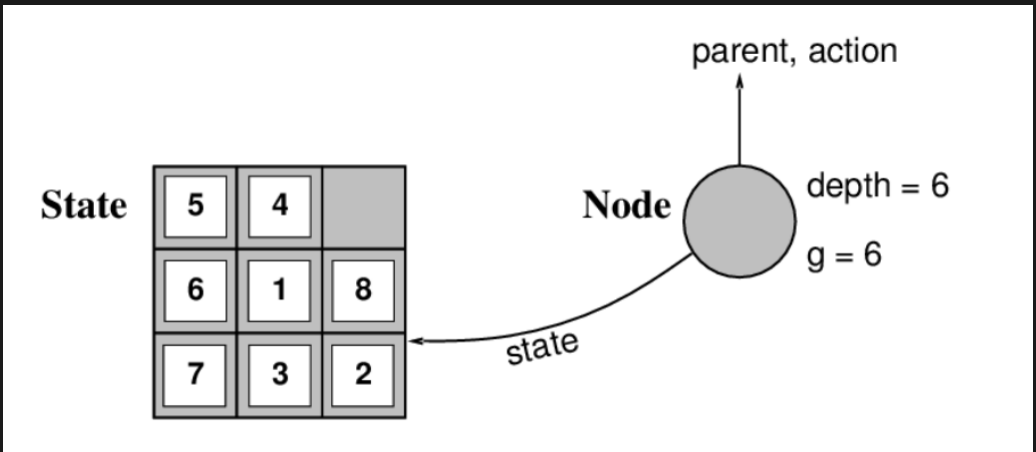


Figura 2:

1.8 General Tree Search

```
function Tree-Search(problem,frontier) returns a solution or failure
frontier <- Inser(Make-Node(problem.Initial-State))
while not isEmpty(frontier) do
      node <- Pop(frontier)
      if problem.Goal-Test(node.State) then return node
      frontier <- InsertAll(Expand(node,problem))
end loop
return failure
```

1.9 Implementation: Expand Nodes

```
function Expand(node,problem) return a set of nodes
successors <- the empty set
for each action, result in Successor-Fn(problem,node.State) do
    s <- a new Node
    s.Parent.Node <- node;
    s.Action <- action
    s.State <- result
    s.Path-Cost <- node.Path-Cost + Ste-Cost(node.State,action,result)
    s.Depth <- node.Depth+1
    add s to successors
return successors
```

Ovviamente per la ricerca si applicano delle strategie in base a completezza di tempo, di problema, di spazio, di ottimalità.

La complessità di tempo e spazio è misurata da:

- b: maximum branching factor of the search tree
- d: depth of the least-cost solution
- m: maximum depth of the state space (may be ∞)

1.10 Graph Search

```
function Grapth-Search(problema,frontier) return a solution, or failure
explore <- an empty set
frontier <- Insert(Make-Node(problem.Initial-State))
while not IsEmpty(frontier) do
      node <- Pop(frontier)
      if problem.Goal-Test(node.State) the return node
      if node.State is not in explored then
        add node.State to explored
        frontier <- InsertAll(Expand(node,problem)
      end if
end loop
return failure
```

2 Local Search

2.1 Iterative improvement algorithms

Nella maggior parte dei problemi di ottimizzazione, il percorso è irrilevante, ciò che conta è raggiungere l'obiettivo.

Lo state space è un set completo di configurazioni.

2.2 Hill-Climbing

```
function Hill-Climbing(problem) returns a state that is a local maximum
         inputs: problem, a problem
         local variables: current, a node
                          neighbor, a node

         current <- Make-Node(problem.Initial-State)
         loop do
              neighbor <- a highest-valued successor of current
              if neighbour.Value <= current.Value then return
                 current.State
              end if
              current <- neighbor
         end

function Simulated-Annealing(problem,schedule) returns a solution state
         inputs: problem, a problem
                 schedula, a mapping fromm time to "temperature"
```

```
local variables: current, a node
                 next, a node
                 T, a "temperature" controlling prob. of
                    downward steps

current <- Make-Node(problem.Initial-State)
for t <- 1 to infty do
    T <- schedule(t)
    if T = 0 then return current
    next <- a random selected successor of current
    \Delta E <- next.Value - current.Value
    if \Delta E > 0 then current <- next
    else current <- next only with probability $e^{\frac{\
       Delta E}{T}}$
```

3 Local Search in continuous spaces

Example

Placing airports in Romania

Si vogliono posizionare tre aeroporti per minimizzare la somma del quadrato delle distanze tra ogni città e l'aeroporto più vicino.

- Airport location: $(x_1, y_1), (x_2, y_2), (x_3, y_3)$
- City location: (x_c, y_c)
- C_a cities closest to airport a $a = \{1, 2, 3\}$

Minimizzazione oggettiva:

$$\sum_a \sum_{c \in C_a} (x_a - x_c)^2 + (y_a - y_c)^2$$

3.1 Discretizzazione

È un metodo che trasforma uno spazio continuo in uno spazio discreto, così da poter applicare gli algoritmi di ricerca locale.

- fix δ
- si considera una variazione di $\pm\delta$ in ogni cordinata
- si utilizza un qualsiasi algoritmo di ricerca locale discreto

La prima scelta ricade sull'algoritmo di Hill-Climbing fino a quando una variazione casuale non migliora lo stato corrente.

Si utilizza il simulated annealing per evitare di rimanere bloccati in un massimo locale.

Si calcola infine il gradiente di $f(X)$ analiticamente ($X = \{x_1, \ldots, x_n\}$)

3.2 Calcolo del gradiente

$\nabla f(X) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n}\right)$

Sull'esempio dell'aeroporto, si calcola come segue: $f(X) = \sum_a \sum_{c \in C_a} (x_a - x_c)^2 + (y_a - y_c)^2$

e $X = \{x_1, y_1, x_2, y_2, x_3, y_3\}, n = 6$

- $\frac{\partial f}{\partial x_1} = \sum_{c \in C_1} 2(x_1 - x_c)$
- per trovare l'estremo si risolve per $\nabla f(X) = 0$
- risolvere in locale

3.3 Local gradient

Spesso non è possibile risolvere $\nabla f(X) = 0$ in maniera analitica globalmente, ma si può risolvere localmente.

- Si prova a proseguire il calcolo per ogni coordinata in direzione positiva salendo lungo il gradiente.
- Più è alta la pendenza (quindi più la derivata è grande), più è probabile che la direzione sia quella giusta.

Si consideri una funzione generica a due variabili $g(x_1, x_2)$, la si aggiorni per il metodo del gradiente.

$x_1 \leftarrow x_1 + \alpha \times \frac{\partial g(x_1, x_2)}{\partial x_1}, x_2 \leftarrow x_2 + \alpha \times \frac{\partial g(x_1, x_2)}{\partial x_2}$

Si proceda con la notazione vettoriale

$$X = [x_1, x_2]^T, \nabla g(X) = [\frac{\partial g}{\partial x1}, \frac{\partial g}{\partial x2}]^T$$
$$X \leftarrow X + \alpha \times \nabla g(X)$$

α è la grandezza del passo, una costante positiva di basso valore, è fondamentale trovare un valore ottimale per α.

Se α è troppo grande, si rischia di saltare alcune soluzioni ottimali, se è troppo piccolo, si rischia di effettuare troppi passi e impiegare troppo tempo per raggiungere la soluzione.

3.4 Newton Raphson

Il metodo di Newton-Raphson è un metodo iterativo per trovare le radici di diversi problemi di local search in uno spazio continuo.

La tecnica generica consiste nel trovare le radici di una funzione $f(x)$, quindi si risolve $f(x) = 0$.

Scegliere un valore iniziale di $\overline{x}_0$

Aggiornare progressivamente la stima di $\overline{x}_{n+1} = \overline{x}_n - \frac{g(\overline{x_n})}{g'(\overline{x_n})}$

dove $g'(x) = \frac{\partial g(x)}{dx}$ è la derivata della funzione $g(x)$.

Aplpicazione del metodo di Newton-Raphson

Si consideri la funzione $g(x) = x^2 - a$

Si dimostri che il metodo di Newton porta alla soluzione ricorsiva cos' definita:

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{a}{x_n}\right)$$

Si imposti $a = 4, x_0 = 1$ e si calcoli x_i con $i \in \{1, 2, 3\}$

□

3.5 Interpretazione geometrica del metodo di Newton

Si assuma $y = f(x)$ con radice $x = r$ che sia tangente al punto $(a, f(a))$ con funzione $y = f(a) + (x - a)f'(a)$

Sia b la intercetta per la retta tangente, $0 = f(a) + (x - a)f'(a)$ si ricava: $b = a - \frac{f(a)}{f'(a)}$

b è la nuova stima della radice per il metodo di Newton-Raphson partendo da a.

Per trovare un estremo per f(X) bisogna trovare $\bar{(}X)$ tale che $\nabla f(\overline{X}) = 0$

È possibile utilizzare il metodo di Newton-Raphson dove $g(X)$ è la derivata di $f(X)$, quindi $\nabla f(X)$

In forma di matrice vettoriale, l'aggiornamento di Newton-Raphson diventa:

$$x \leftarrow x - H_f^{-1}(x)\nabla f(x)$$
$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

Per problemi di grandi dimensioni, calcolare n^2 valori della matrice Hessiana e invertire la matrice $N \times N$ potrebbe essere computazionalmente costoso. Si utilizzano quindi approsimazioni della matrice Hessiana.

A volte non si ha a disposizione la funzione $f(X)$. In quel caso si stima la pendenza sperimentalmente, calcolando le coordinate su valori diversi di $\pm\delta$ studiando come varia la funzione (in pratica un hill-climbing).

4 CSP, Contrait Satisfaction Problem

Assunzioni: singolo agente, azioni deterministiche, stato monitorabile, spazio discreto.

Planning

- Si vuole conoscere il percorso per raggiungere l'obiettivo,
- i percorsi hanno un costo associato,
- le euristiche specificano una guida alla risoluzione del probkema.

Identification

- L'unico elemento di interesse è la soluzione finale,
- la local search potrebbe tornare parecchio utile (ma la sua efficacia non è garantita), e può essere utilizzata anche in spazi continui.
- la CSP è una classe specializzata nell'identificazione dei problemi.

Un problema di ricerca standard può essere descritto in tal maniera:

- Lo stato iniziale è definito da un qualsiasi insieme di variabili e valori (scatola nera),
- Il goal test può essere una qualsiasi funzione tra gli stati,
- La funzione di transizione (che mappa la funzione successiva) può essere una qualsiasi funzione che mappa lo stato corrente in un set di stati successivi.
- Uno stato è definito da un set di variabili $X = X_i$ che possono assumere valori da un dominio $D = D_i$
- Il goal test è un set di vincoli che specificano le combinazioni di valori ammissibili per un sottoinsieme di variabili,
- Si possono utilizzare strategie di risoluzione generiche che sfruttano queste caratteristiche che risultano più efficienti ed efficaci delle strategie tradizionali.

Example

Si piazzino N regine su una scacchiera con $N \times N$ caselle in maniera che nessuna regina possa attaccare un'altra regina.

- $X : \{x_{i,j}\}, i \in \{1, \ldots, N\}, j \in \{1, \ldots, N\}$: si ottiene così una sola variabile per ogni cella della scacchiera
- $D : \{D_{i,j}\}, D_i \in 0, 1$: dominio di valori ammissibili booleano (0=false, 1=true)
- $\forall i, j, k (X_{i,j}, X_{i,k}) \in \{(0, 0), (0, 1), (1, 0)\}$ soltanto una regina per ogni riga.
- $\forall i, j, k (X_{i,j}, X_{k,j}) \in \{(0, 0), (0, 1), (1, 0)\}$ soltanto una regina per ogni colonna.
- $\forall i, j, k (X_{i,j}, X_{i+k,j+k}) \in \{(0, 0), (0, 1), (1, 0)\}$ soltanto una regina su ogni diagonale (a dx)
- $\forall i, j, k (X_{i,j}, X_{i+k,j-k}) \in \{(0, 0), (0, 1), (1, 0)\}$ soltanto una regina su ogni diagonale (a sx)
- $\sum_{i,j} X_{i,j} = N$

Si consideri solo una regina per ogni colonna

- X: $\{x_i\}, i \in \{1, \ldots, N\}$: si ottiene così una sola variabile per ogni colonna della scacchiera
- D: $\{D_i\}, D_i \in \{1, \ldots, N\}$: riga per la regina nella colonna i
- $C : \{C_1, \ldots, C_k\}, k \in \{1, \ldots, Nx(N-1)/2\}$ un vincolo per ogni coppia di variabili, che specifica le posizioni ammissibili delle due regine.

Per esempio

Questa formulazione rende alcuni vincoli impliciti, non è ad esempio possibile assegnare due regine alla stessa colonna, non bisogna eseguire un controllo.

- $C_1 : (x_1, x_2) \in \{(1,3),(1,4)(2,4),(3,1),(4,1),(4,2)\}$
- $C_2 : (x_1, x_3) \in \{(1,2),(1,4),(2,1),(2,3),(2,4),(3,2),(3,4),(4,2),(4,3)\}$
- ...

Figura 3:

Problemi di combinatoria

Dato un set di possibili soluzioni, si cerca di trovare la migliore possibile. Le soluzioni sono tante, ma discrete e finite, una ricerca completa non è possibile.z

5 Logical Agents

Knowledge base = è un insieme di frasi scritte in linguaggio formale